

IV

Consensus Guide

Nakamoto consensus: the double-spend race, the arithmetic of confirmations,
and the deployed most-work rule

Marek
m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs Nakamoto consensus and quantifies its security. It assumes the probability foundations of the *Math Guide* and the hash and commitment foundations of the *Crypto Guide*. It states the block tree and the most-work rule as the protocol specification defines them and as the deployed zebra implements them; constructs the race-specific probability that the analysis needs (memoryless clocks, the biased random walk, the negative binomial law); develops the exact first-seen-model double-spending analysis of Rosenfeld's *Analysis of hashrate-based double-spending* (the catch-up theorem, the confirmation formula, and the economics of the attack), with every derived number recomputed by script and the results instantiated for Zcash's proposed 25-second blocks; describes the draft NU7 transition to 25-second blocks, fee removal, and halving-preserving reissuance; and describes the protocol and implementation safeguards (difficulty adjustment, reorg windows, coinbase maturity, wallet confirmation policy) and their relation to the model's assumptions. Where the specification, the paper, and the deployed nodes disagree, the divergence is stated.

Contents

1	Scope, sources, and the two double-spends	4
1.1	Sources and their standing	4
2	The block tree and the most-work rule	5
2.1	Work, and the best chain	5
2.2	The attack	6
3	Probabilistic preliminaries	7
3.1	Memoryless clocks and the attribution sequence	7
3.2	The negative binomial law	8
4	The catch-up probability	9
5	Waiting for confirmations	11
6	Confirmation requirements	14
6.1	Confirmation times at 25-second spacing	15
7	The economics of the attack	17
7.1	Reward-normalised exposure	18
8	NU7: 25-second blocks and bounded shielded work	19
8.1	Chain selection and the tie-break	20
8.2	Difficulty adjustment	20
8.3	Equihash and the memoryless model	21
8.4	Rollback limits, coinbase maturity, and finality	21
8.5	Wallet confirmation policy	22
8.6	A block budget for shielded work	23
8.7	Scheduled issuance with shorter blocks	24
8.8	The NSM reserve and delayed reissuance	25
9	Layer boundary	27

1 Scope, sources, and the two double-spends

This volume gives a quantitative account of Nakamoto consensus (proof of work, the most-work rule, and the probabilistic finality they provide), the mechanism that selects the ledger the protocol layers above it assume. The analysis follows Meni Rosenfeld’s *Analysis of hashrate-based double-spending* (11 December 2012; latest version 12 February 2014; arXiv:1402.2009), an exact counterpart to the approximate analysis in Satoshi Nakamoto’s whitepaper; we reprove its results in full, recompute its tables by script, and instantiate its conclusions for the proposed NU7 parameters.

The term “double-spending” names two distinct attacks, of which this volume treats one. Two spends of the same shielded note reveal the same nullifier, and the protocol specification’s §“Nullifier Sets” forbids a repeated nullifier within a transaction or valid block chain; such a same-chain attempt is therefore invalid under the consensus rules. Consensus must prevent the chain-level attack: the attacker pays a merchant on the public chain, then replaces that chain with another in which the payment does not occur. No zero-knowledge proof (*Crypto Guide*, §“Interactive proofs, zero knowledge, and SNARKs”) excludes this attack; under the model of this volume, the proof-of-work race bounds its probability of success.

The analysis assumes the *Math Guide*’s probability section (§“Probability, asymptotics, and computation”), which constructs probability from its definition (finite spaces, conditioning, independence, random variables, expectation) and makes the three extensions used here: countable additivity, continuity along monotone events, and density-defined laws (§“Beyond finite spaces: countable additivity, limits, and densities”). The block and anchor vocabulary uses the *Crypto Guide*’s hash and commitment sections. Section 3 constructs the race-specific instruments that no lower volume constructs: memoryless clocks, random walks, and the negative binomial law; the algebra used is elementary. The results can be read from Sections 2, 6, and 8 alone, without the proofs.

1.1 Sources and their standing

Rosenfeld’s analysis supplies the probability and economic model, not consensus rules; its two printed tables are reproduced by exact arithmetic before the model is instantiated for Zcash. The protocol specification and the relevant ZIPs define the protocol.

This volume targets the *draft NU7 rules* as of 6 October 2026: ZIP-259 selects ZIP-218, ZIP-235, ZIP-237, ZIP-2003, and three funding-stream amendments: Revision 2 of ZIP-207, Revision 3 of ZIP-214, and ZIP-2008. The activation height is assigned on Testnet, the test network, but not on Mainnet, the production network, so the Mainnet heights derived from it are unassigned. These gaps are stated where they affect a calculation; no unassigned constant is treated as a deployed rule.

Remark 1.1 (Defects of the arXiv text). The arXiv v1 PDF of the paper contains exactly two citations, both rendered as “[?]” (in the abstract and in §1), and no References section. Both refer, by context, to the Nakamoto whitepaper, and we read them so. The paper numbers only two of its displayed equations (its equation (1) is our Theorem 5.2, its equation (2) our equation (3)) and sets its two model assumptions as a bare numbered list; the *Assumption* environments of Section 4

are ours.

2 The block tree and the most-work rule

Zcash transactions are grouped into blocks, and every block other than the genesis block names its parent by including the parent header’s hash (*Crypto Guide*, §“Hash functions and the random oracle model”). Blocks therefore form a tree rooted at the genesis block, and each root-to-leaf path, a *branch*, is one internally consistent history. A branch may contain a transaction that conflicts with a transaction in a sibling branch; the attack that this volume quantifies exploits such conflicts. This section defines the work of a block, the best valid block chain, and confirmations, then states the attack.

2.1 Work, and the best chain

Nodes select one history by a rule that each applies locally, and they converge once the rule strictly prefers one valid branch. The rule weighs each block by the computational effort that its header target represents, rather than counting blocks as a “longest” rule would.

Definition 2.1 (Work of a block). Let `ToTarget` be the conversion from the header’s `nBits` field to its 256-bit target threshold (specification §7.7.4). The *work* of a block is

$$\text{work} = \left\lfloor \frac{2^{256}}{\text{ToTarget}(\text{nBits}) + 1} \right\rfloor$$

(specification §7.7.5, “Definition of Work”). A lower target admits fewer valid headers, so a block meeting it certifies proportionally more expected hashing; work is the additive measure of that expected hashing.

The deployed Zebra validator computes this quantity without leaving 256-bit arithmetic, as one plus the integer quotient of the bitwise complement of the target by one more than the target; the retired `zcashd` computed the identical expression. This computation agrees with Definition 2.1 exactly: writing t for the target and $A = 2^{256}$, the complement is $\neg t = A - 1 - t$, and

$$\left\lfloor \frac{A - 1 - t}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A - (t + 1)}{t + 1} \right\rfloor + 1 = \left\lfloor \frac{A}{t + 1} \right\rfloor,$$

an identity also checked by script over random targets.

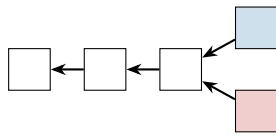
Definition 2.2 (Best valid block chain). Specification §3.3 (“The Block Chain”): a node “sums the work, as defined in §7.7.5, of all blocks in each valid block chain, and considers the valid block chain with greatest total work to be best. To break ties between leaf blocks, a node will prefer the block that it received first.”

Rosenfeld’s paper states the same rule for Bitcoin, “the branch representing the most proof of work”, with first-seen tie-breaking, and the model adopts it. The shorthand “longest chain” is exact only while difficulty is constant, which is the paper’s Assumption 4.2 and not a property of the deployed chain (Section 8.2). The rule marks no chain as permanently chosen: a node that

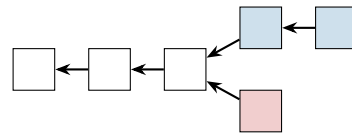
learns of a heavier valid branch switches to it however deep the fork point, subject only to the node-local rollback limits of Section 8.4.

Definition 2.3 (Confirmations). A transaction has n *confirmations* if it is included in a block of the best chain and there are n blocks on the path from that block to the chain’s leaf, inclusive. The block containing the transaction is its first confirmation.

Different nodes may briefly disagree on the best chain when two blocks of equal cumulative work arrive in different orders. Absent a concurrent extension of the other branch, a later block makes one branch strictly heavier, and nodes converge on it once that block propagates (Figure 1). Such ties arise without an attacker. In the attack of Section 5, the attacker instead builds a fork in private and releases it only once it carries more work than the public branch.



(a) tied branches: nodes disagree



(b) one more block makes the upper branch heavier

Figure 1. Branch selection. Arrows point from child to parent. In panel (a) two blocks extend the same parent and the branches carry equal work; each node keeps the one it saw first. In panel (b) a new block lands on the upper branch, which now has strictly more cumulative work; during propagation, every node that knows this state selects the upper branch, unless the lower branch has meanwhile been extended. At constant difficulty “more work” and “longer” coincide, which is how the paper, and this volume’s figures, may draw block counts.

2.2 The attack

The paper (§3) describes the double-spend attack as follows. The attacker:

1. broadcasts a transaction paying the merchant;
2. mines in secret a branch from the last block before that transaction, containing instead a conflicting transaction paying the attacker;
3. waits until the merchant’s transaction has n confirmations and the merchant hands over the goods;
4. in the paper’s first-seen tie model, extends the private branch until it carries more work than the public one, then broadcasts it. Every node that receives it switches to it, the payment to the merchant leaves the best chain, and the attacker holds both the goods and the coins.

Step 4 violates no validity rule: the released branch is a valid chain, distinct from the public one. The success of the attack is therefore a question of probability alone. Sections 4 and 5 compute exactly, in the first-seen model, the probability that the private branch ever carries more work than the public one.

3 Probabilistic preliminaries

This section constructs the probabilistic tools of the double-spend analysis in Sections 4 to 6: exponential clocks, which model block discovery; the attribution sequence (Proposition 3.4), which reduces the race in continuous time to a sequence of Bernoulli trials; the negative binomial law of the attacker’s progress (Proposition 3.5); and the normalisation of a first-to- n race (Lemma 3.6). Probability spaces, conditioning, independence, random variables and expectation are those of the *Math Guide*, §“Probability, asymptotics, and computation”; countable additivity, continuity along monotone events, density-defined laws and infinite sequences of independent trials are those of its §“Beyond finite spaces: countable additivity, limits, and densities”.

3.1 Memoryless clocks and the attribution sequence

Definition 3.1 (Exponential clock). A random variable X is *exponential with rate $\lambda > 0$* if $\Pr[X > t] = e^{-\lambda t}$ for all $t \geq 0$ — a law defined by the density $\lambda e^{-\lambda t}$ on $t \geq 0$ in the sense of the *Math Guide*’s density definition, with mean $1/\lambda$.

Lemma 3.2 (Memorylessness). *An exponential X satisfies $\Pr[X > s + t \mid X > s] = \Pr[X > t]$ for all $s, t \geq 0$: the remaining wait of a clock that has not fired has the law of a fresh clock.*

Proof. $\Pr[X > s + t \mid X > s] = e^{-\lambda(s+t)} / e^{-\lambda s} = e^{-\lambda t}$. □

Lemma 3.3 (Race of two clocks). *Let X and Y be independent exponentials with rates λ and μ . Then*

$$\Pr[X < Y] = \frac{\lambda}{\lambda + \mu},$$

and the winning time $\min(X, Y)$ is exponential with rate $\lambda + \mu$.

Proof. Integrating over the time at which X fires,

$$\Pr[X < Y] = \int_0^\infty \lambda e^{-\lambda t} \Pr[Y > t] dt = \int_0^\infty \lambda e^{-(\lambda+\mu)t} dt = \frac{\lambda}{\lambda + \mu}.$$

For the minimum, $\Pr[\min(X, Y) > t] = \Pr[X > t] \Pr[Y > t] = e^{-(\lambda+\mu)t}$. Moreover, the joint density that X wins at time t factors as

$$\lambda e^{-(\lambda+\mu)t} = \frac{\lambda}{\lambda + \mu} (\lambda + \mu) e^{-(\lambda+\mu)t},$$

and similarly for Y . Thus the winner is independent of the winning time. □

Proposition 3.4 (The attribution sequence). *Model the honest network and attacker as independent exponential clocks with rates p/T_0 and q/T_0 , restarting when either fires (Section 4). Successive block attributions form an infinite sequence of independent, identically distributed Bernoulli trials in the *Math Guide*’s sense: attacker with probability q , honest network with probability p , independent of all earlier attributions and all elapsed times.*

Proof. By Lemma 3.3 the first block is the attacker's with probability $(q/T_0)/(p/T_0 + q/T_0) = q$. When a block is found, the finder's clock restarts by construction, and the loser's remaining wait is distributed as a fresh clock by Lemma 3.2; the state after each block is therefore probabilistically identical to the start, independent of the past. The density factorisation in Lemma 3.3 also makes each winner independent of that race's elapsed time. The claim follows by induction. $\square$

Proposition 3.4 is the paper's reduction (its §3, with the clock rates of its footnote 1) of the race in continuous time to a sequence of trials. Whether the attacker's branch ever overtakes the public chain depends only on the *order* of block discoveries, not on their times; every result below is therefore a statement about a p/q trial sequence, and the time constant T_0 does not appear in any of them.

3.2 The negative binomial law

Proposition 3.5 (Progress of the loser). *In an attribution sequence with attacker probability $q = 1 - p$, let m be the number of attacker blocks found strictly before the honest network's n -th block. Then*

$$P(m) = \binom{m+n-1}{m} p^n q^m, \quad m = 0, 1, 2, \dots$$

— the negative binomial distribution.

Proof. The event constrains the first $m+n$ trials as follows: the $(m+n)$ -th trial is honest (the n -th honest success), and among the first $m+n-1$ trials exactly m are the attacker's, in any of $\binom{m+n-1}{m}$ orders. Each such order has probability $p^{n-1} q^m \cdot p$. $\square$

Lemma 3.6 (First-to- n race). *In the same sequence, let H_n be the event that the honest network reaches n blocks before the attacker does, and A_n the reverse. Then H_n and A_n partition the sample space, and*

$$\Pr[H_n] = \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^n q^m, \quad \Pr[A_n] = \sum_{b=0}^{n-1} \binom{b+n-1}{b} q^n p^b,$$

$$\Pr[H_n] + \Pr[A_n] = 1.$$

Proof. Among the first $2n-1$ trials one side already has n successes, so the race terminates; a tie is impossible because block counts advance one at a time. The event H_n says the n -th honest block arrives while the attacker holds $m \leq n-1$; summing Proposition 3.5 over those m gives the first sum, and A_n is the same computation with the roles of p and q swapped. $\square$

Remark 3.7 (From hash trials to exponential clocks). The exponential model is a limit, not an axiom. A miner's work is modelled as a sequence of candidate evaluations, each an independent Bernoulli trial with a small success probability; the number of trials to the first success is geometric, and a geometric law with small success probability, rescaled by its mean, converges to the

exponential of Definition 3.1. The limit requires *progress-freeness*: a failed candidate carries no information usable by the next one. Whether Zcash’s Equihash satisfies this is examined with the deployed parameters in Section 8.3.

4 The catch-up probability

When the merchant of Section 2.2 ships the goods, the attacker’s private branch is some number of blocks behind the public chain; the attack succeeds if that branch ever overtakes it. This section models the deficit as a random walk (Definition 4.3) and computes the probability of that event as a function of the deficit (Theorem 4.4); the next section derives the distribution of the deficit. Both computations rest on the paper’s two assumptions, stated nearly verbatim (the environment form is ours, Remark 1.1):

Assumption 4.1 (Constant hashrates). The total hashrate of the honest network and the attacker is constant. Combined they have a hashrate of H , of which pH belongs to the honest network and qH to the attacker, where $0 < p, q < 1$ and $p + q = 1$.

Assumption 4.2 (Constant difficulty). The mining difficulty is constant, and such that with hashrate H the average time to find a block is T_0 .

Under Assumption 4.2 every block carries equal work, so the branch with the most work is the longest, and the race is measured in blocks. The deployed chain departs from both assumptions: difficulty adjusts after every block and hashrate drifts. The formulas are therefore an equal-difficulty baseline, not an exact model of deployment (Section 8.2).

Definition 4.3 (The deficit walk). Let z denote the honest chain’s lead in blocks over the attacker’s branch. By Proposition 3.4, each newly found block moves z up by 1 with probability p (honest block) or down by 1 with probability q (attacker block), independently of the past:

$$z_{i+1} = \begin{cases} z_i + 1 & \text{with probability } p, \\ z_i - 1 & \text{with probability } q. \end{cases}$$

The attack succeeds when z reaches -1 : the private branch is then strictly ahead, and once released it replaces the public chain, removing the payment to the merchant.

Theorem 4.4 (Catch-up probability; Rosenfeld §3). Let a_z be the probability that the walk of Definition 4.3, started at deficit z , ever reaches -1 . Then

$$a_z = \min(q/p, 1)^{\max(z+1, 0)} = \begin{cases} 1 & \text{if } z < 0 \text{ or } q \geq p, \\ (q/p)^{z+1} & \text{if } z \geq 0 \text{ and } q < p. \end{cases} \quad (1)$$

Proof. For $z < 0$ the walk has already succeeded, so $a_z = 1$; assume $z \geq 0$. Fix an integer $N > z$ and absorb the walk at both -1 and N ; let $b_N(z)$ be the probability of reaching -1 before N .

Conditioning on the first step,

$$h_N(z) = p h_N(z+1) + q h_N(z-1) \quad (0 \leq z \leq N-1), \quad h_N(-1) = 1, \quad h_N(N) = 0,$$

a second-order linear recurrence. The trial solution $h_N(z) = t^z$ satisfies it exactly when t is a root of the *characteristic polynomial* $pt^2 - t + q = p(t-1)(t-q/p)$.

If $q \neq p$ the roots 1 and q/p are distinct, so $h_N(z) = A + B(q/p)^z$; the boundary conditions give

$$h_N(z) = \frac{(q/p)^z - (q/p)^N}{(q/p)^{-1} - (q/p)^N}.$$

If $q = p$ the root is double and $h_N(z) = A + Bz$, giving $h_N(z) = (N-z)/(N+1)$.

Now let $N \rightarrow \infty$. The events $E_N = \{\text{reach } -1 \text{ before } N\}$ are nondecreasing in N , and their union is the event that the walk ever reaches -1 : a walk that does so does it at a finite time, having visited finitely many states, so E_N holds for every N above the largest state visited. By continuity along monotone events (*Math Guide*, §“Beyond finite spaces: countable additivity, limits, and densities”), $a_z = \lim_{N \rightarrow \infty} h_N(z)$. For $q < p$ the term $(q/p)^N$ vanishes and the quotient tends to $(q/p)^{z+1}$; for $q = p$, $(N-z)/(N+1) \rightarrow 1$; for $q > p$, dividing numerator and denominator by $(q/p)^N$ sends both to -1 , so the quotient tends to 1. $\square$

Corollary 4.5 (Half or more always catches up). *When $q \geq p$, the attacker overtakes the public chain with probability 1 from any finite deficit. No confirmation count protects a payment when the attacker controls at least half of the hashrate.*

Corollary 4.6 (Geometric decay in the deficit). *When $q < p$, each additional block of deficit multiplies the catch-up probability by q/p . Concretely (script-computed): at $q = 10\%$, $a_0 = 1/9 \approx 0.1111$ and $a_5 \approx 1.88 \times 10^{-6}$; at $q = 30\%$, $a_0 = 3/7 \approx 0.4286$ and $a_5 \approx 6.20 \times 10^{-3}$; at $q = 45\%$, $a_0 = 9/11 \approx 0.8182$ and $a_5 \approx 0.300$. Near one half the factor q/p approaches 1, and a deficit of five blocks still leaves the attacker a 30% chance.*

Corollary 4.7 (Blocks, not time). *Equation (1) contains neither T_0 nor any other time scale: the security of a deficit is a function of block counts alone. Two chains with different block spacings but the same q offer identical security per confirmation — and therefore different security per minute (Section 6.1).*

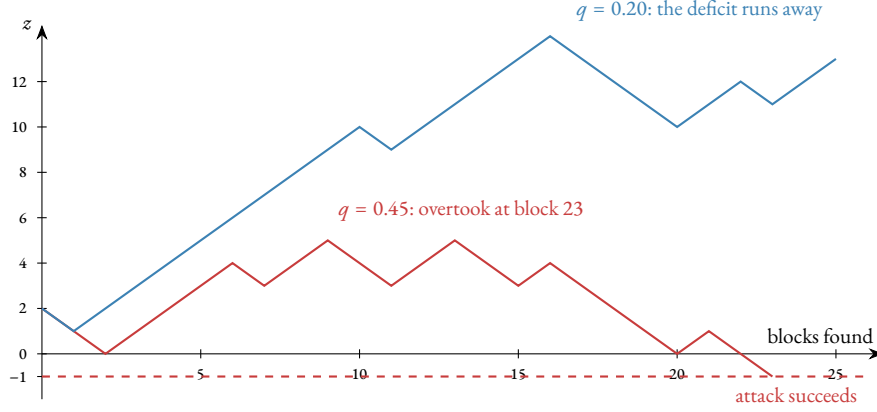


Figure 2. Two runs of the deficit walk from $z = 2$, simulated by script with fixed seeds. At $q = 0.45$ the drift is weak and this run reaches -1 after 23 blocks, so the attack succeeds; at $q = 0.20$ the deficit grows roughly linearly, and by Theorem 4.4 the chance of ever returning from deficit 13 is $(1/4)^{14} \approx 3.7 \times 10^{-9}$.

5 Waiting for confirmations

A merchant defends against a double-spend by shipping only after the paying transaction has n confirmations (Definition 2.3). This section fixes one convention of the paper’s attack timeline and computes, in the paper’s first-seen model, the exact probability that the attacker overtakes after n confirmations.

Remark 5.1 (The pre-mined block). The paper assumes the attacker holds one block before the race is scored: “we assume one block was pre-mined by the attacker before commencing the attack” (§4). The attacker forks from the public leaf at the moment the merchant’s transaction is broadcast, and times the broadcast to coincide with a private block he has found but not yet announced. At the moment the honest chain reaches n confirmations, the attacker who has found m further blocks therefore holds $m + 1$, and the deficit walk of Definition 4.3 starts at

$$z = n - m - 1.$$

The convention is a modelling choice: an attacker with no pre-mined block starts at $z = n - m$, and for a minority attacker every probability below is smaller. It is also the convention under which the two sums in the proof of Theorem 5.2 are equal, so that the final formula describes a symmetric race.

By Proposition 3.5, the number m of attacker blocks found while the honest network finds its n is negative binomial. Averaging the catch-up probability of Theorem 4.4 over m gives the success probability under the first-seen tie rule of the specification and of Rosenfeld, which Zebra also applies from version 6.4.0 (Section 8.1). The model does not give an exact rollback probability for the deployed network.

Theorem 5.2 (Double-spend success probability; Rosenfeld eq. (1)). *Let the merchant wait for $n \geq 1$ confirmations, under Assumptions 4.1 and 4.2 and the convention of Remark 5.1, with*

a private branch accepted only after it becomes strictly heavier. The probability that the attacker's branch eventually overtakes the public chain is $r(q, n) = 1$ if $q \geq p$, and otherwise

$$r(q, n) = 1 - \sum_{m=0}^{n-1} \binom{m+n-1}{m} (p^n q^m - p^m q^n) = 2 \sum_{m=0}^{n-1} \binom{m+n-1}{m} p^m q^n. \quad (2)$$

Proof. Conditioning on m and applying Theorem 4.4 at $z = n - m - 1$,

$$r = \sum_{m=0}^{\infty} P(m) a_{n-m-1} = \underbrace{\sum_{m=0}^{n-1} P(m) \left(\frac{q}{p}\right)^{n-m}}_{\text{behind: must catch up}} + \underbrace{\sum_{m=n}^{\infty} P(m)}_{\text{already ahead}},$$

valid for $q < p$; when $q \geq p$ every $a_{n-m-1} = 1$ and $r = \sum_m P(m) = 1$ immediately. For the first sum, each term transforms exactly:

$$\binom{m+n-1}{m} p^n q^m \left(\frac{q}{p}\right)^{n-m} = \binom{m+n-1}{m} p^m q^n,$$

which by Lemma 3.6 is the probability that the attacker's n -th block arrives while the honest network holds m ; summed over $m \leq n - 1$, the first sum is $\Pr[A_n]$, the probability that the attacker wins the first-to- n race. The second sum is $\Pr[m \geq n] = 1 - \Pr[H_n] = \Pr[A_n]$ by the same lemma. Hence $r = 2\Pr[A_n]$, the right-hand form of (2); the middle form follows from $\Pr[A_n] = 1 - \Pr[H_n]$ applied to one of the two copies. The printed upper limit in the paper's equation (1) is n rather than $n - 1$; the $m = n$ term is identically zero, so the forms agree. $\square$

Remark 5.3 (The race doubling). Under the one-pre-mined-block convention, the *catch-up half* — the honest chain reaches n first and the attacker later overtakes — has exactly the same probability as the *already-ahead half*, in which the attacker reaches n first. A first-to- n winner may have fallen behind and recovered along the way; neither half asserts otherwise. The double-spend probability is therefore twice the probability of winning a first-to- n race. The identity also covers the boundary case: at $q = p$ each race half is exactly $\frac{1}{2}$, giving $r = 1$ with no case split, continuously with the $q > p$ regime. The algebra here is the paper's; reading its two sums as the two halves of one race, and the observation that the pre-mine convention is what makes them equal, are this volume's presentational additions (verified by script over random (q, n) ; the two sums agree to floating-point precision).

Example 5.4 (A full race, exactly). Take $q = 1/5$ (a twenty-percent attacker) and $n = 6$. The deficit starts at $z = 5 - m$, the catch-up factor is $(q/p)^{6-m} = 4^{-(6-m)}$, and every quantity below is an exact rational, printed to six decimals (script-computed):

m	$P(m)$	$a_{5-m} = 4^{-(6-m)}$	product
0	0.262144	0.000244	0.000064
1	0.314573	0.000977	0.000307
2	0.220201	0.003906	0.000860
3	0.117441	0.015625	0.001835
4	0.052848	0.062500	0.003303
5	0.021139	0.250000	0.005285
catch-up half (sum of products)			0.011654
already-ahead half, $\Pr[m \geq 6]$			0.011654
$r(0.2, 6)$			0.023308

The two halves are equal exactly, by Remark 5.3, and the closed form (2) evaluates to the same 0.023308. For a merchant who ships after six confirmations against a twenty-percent attacker, the model gives a 2.33% probability of a strict overtake.

Remark 5.5 (Satoshi’s approximation). The whitepaper’s §11 computes the same quantity with one approximation and two offsetting conventions: the attacker’s progress is approximated as Poisson with mean $\lambda = nq/p$ (the law $\Pr[k] = e^{-\lambda} \lambda^k / k!$ on the nonnegative integers); no block is pre-mined, but success is scored at *breakeven* rather than strictly ahead, and those two conventions cancel exactly — the whitepaper’s catch-up factor for m attacker blocks is $(q/p)^{n-m}$, term for term the factor of Remark 5.1. The two results therefore differ only by the Poisson approximation. In each of the following script-computed comparisons the approximation lowers the estimate: at $q = 10\%$, $n = 6$, the whitepaper’s formula gives 0.0243% against the exact 0.0591% — an understatement by a factor of about 2.4; at $q = 30\%$, $n = 6$: 13.211% against 15.645%; at $q = 10\%$, $n = 10$: 0.0001% against 0.0008%. The direction of the error is not the same for all (q, n) ; the negative binomial model is preferred because it is exact under the stated assumptions.

Table 1. The double-spend success probability $r(q, n)$, in percent, computed by script from Theorem 5.2. On the rows the paper’s Table 1 also prints ($q \in \{2, 10, 20, 30, 40, 48, 50\}\%$), the script reproduces its values cell-for-cell; the 5% and 45% rows are this volume’s additions.

q	$n=1$	2	3	4	5	6	7	8	9	10
2%	4.000	0.237	0.016	0.0011	$8 \cdot 10^{-5}$			$< 10^{-5}$		
5%	10.000	1.450	0.232	0.039	0.0066	0.0012	$2.1 \cdot 10^{-4}$	$4 \cdot 10^{-5}$	$< 10^{-5}$	
10%	20.000	5.600	1.712	0.546	0.178	0.059	0.020	0.0067	0.0023	0.0008
20%	40.000	20.800	11.584	6.669	3.916	2.331	1.401	0.848	0.516	0.316
30%	60.000	43.200	32.616	25.207	19.762	15.645	12.475	10.003	8.055	6.511
40%	80.000	70.400	63.488	57.958	53.314	49.300	45.769	42.621	39.787	37.218
45%	90.000	85.050	81.375	78.342	75.716	73.375	71.252	69.299	67.487	65.793
48%	96.000	94.003	92.508	91.264	90.177	89.201	88.307	87.478	86.703	85.972
50%					100 for every n					

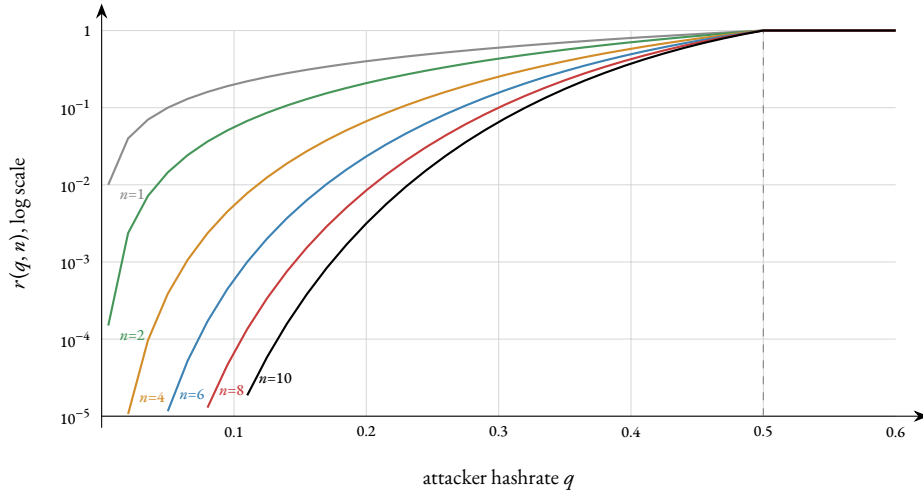


Figure 3. The success probability $r(q, n)$ against the attacker's hashrate share q , for $n \in \{1, 2, 4, 6, 8, 10\}$ confirmations, logarithmic vertical scale clipped at 10^{-5} ; coordinates computed by script from Theorem 5.2. Every curve reaches 1 at $q = \frac{1}{2}$ (dashed) and stays there: to the right of that line, confirmations give no protection. To its left, each added confirmation divides r by a roughly constant factor — the curves are near-equally spaced in log scale — and the factor grows as q falls.

6 Confirmation requirements

Theorem 5.2 reduces the double-spend success probability to the function $r(q, n)$ of the attacker's share and the confirmation count, tabulated in Table 1 and plotted in Figure 3. This section tabulates the confirmation counts required for three risk targets, proves that no finite count has zero risk (Proposition 6.1) and that the required count diverges as q approaches $\frac{1}{2}$ (Proposition 6.2), and converts confirmation counts into wall-clock time at the Zcash block spacing.

Proposition 6.1 (No finite count is final). *For every $0 < q < p$ and every n , $r(q, n) \geq 2q^n > 0$.*

Proof. By Theorem 5.2, $r = 2\Pr[A_n]$, and one way for the attacker to win the first-to- n race is to find the first n blocks outright, an event of probability q^n . $\square$

Proposition 6.2 (Divergence at the majority line). *Fix a target $0 < \varepsilon < 1$. For $0 < q < \frac{1}{2}$, let $n_\varepsilon(q)$ be the least n with $r(q, n) < \varepsilon$. Then $n_\varepsilon(q) \rightarrow \infty$ as $q \uparrow \frac{1}{2}$.*

Proof. First, $n_\varepsilon(q)$ exists. The event A_n that the attacker reaches n blocks first is the event that at least n of the first $2n - 1$ block discoveries are the attacker's. Since $q < p$, each such length- $(2n - 1)$ sequence has probability at most $q^n p^{n-1}$, and there are fewer than 2^{2n-1} sequences. Hence

$$r(q, n) = 2\Pr[A_n] \leq \frac{(4pq)^n}{p} \rightarrow 0,$$

because $4pq < 1$.

Table 2. The least number of confirmations for which $r(q, n)$ falls below each of three targets, computed by script. The count grows logarithmically in the inverse target risk and without bound as q approaches 50%: at the strictest target a 10% attacker requires 6 confirmations, a 45% attacker 539, and a 50% attacker no finite number.

q	$r < 10\%$	$r < 1\%$	$r < 0.1\%$
2%	1	2	3
5%	2	3	4
8%	2	3	5
10%	2	4	6
15%	3	6	9
20%	4	8	13
25%	5	12	20
30%	9	20	32
35%	15	36	58
40%	34	82	133
45%	135	331	539

For fixed n the function $q \mapsto r(q, n)$ is a polynomial on $[0, \frac{1}{2}]$, hence continuous, and $r(\frac{1}{2}, n) = 1$: at $q = p$ each half of the race of Remark 5.3 has probability exactly $\frac{1}{2}$ by Lemma 3.6 and symmetry. Given any N , for each $1 \leq n \leq N$ there is a $\delta_n > 0$ with $r(q, n) > \varepsilon$ whenever $q > \frac{1}{2} - \delta_n$. Taking $\delta = \min_{1 \leq n \leq N} \delta_n > 0$ excludes every $n \leq N$ throughout that neighbourhood, so $n_\varepsilon(q) > N$. Since N was arbitrary, the claimed divergence follows. $\square$

Remark 6.3 (Nothing is special about six). The paper (§5): “There is nothing special about the default, often-cited figure of 6 confirmations. It was chosen based on the assumption that an attacker is unlikely to amass more than 10% of the hashrate, and that a negligible risk of less than 0.1% is acceptable. Both these figures are arbitrary” — six confirmations are “overkill for casual attackers, and at the same time powerless against more dedicated attackers”. Table 2 replaces that figure: a policy fixes the attacker share to be withstood and the acceptable risk, and the table gives the corresponding n . The pair (q, ε) is chosen by policy; the model does not determine it.

6.1 Confirmation times at 25-second spacing

By Corollary 4.7, the success probability depends on the confirmation count, not on elapsed time. The wall-clock duration of a count is set by the target spacing: 25 seconds in ZIP-218. While the attacker mines in private, only the honest share of the hashrate extends the public chain, and the merchant’s wait lengthens accordingly.

Proposition 6.4 (Expected wait for n confirmations). *Under Assumptions 4.1 and 4.2, let the attacker withhold all his blocks from the public chain, and let the first public block found after a transaction’s broadcast include it. The time S_n from the broadcast to the transaction’s n -th confirmation has expectation*

$$\mathbb{E}[S_n] = \frac{nT_0}{p}.$$

Proof. Only honest blocks extend the public chain, and the honest network’s clock has rate $\lambda = p/T_0$ (Proposition 3.4). At the broadcast, and after each attacker block, that clock has not fired, so by Lemma 3.2 its remaining wait has the law of a fresh clock. The restarts of Proposition 3.4 therefore leave the honest blocks, counted from the broadcast, the arrivals of an exponential clock of rate λ in the sense of the *Math Guide*, §“Exponential clocks, Poisson counts, and the Chernoff method”. By Definition 2.3 the n -th confirmation is the n -th arrival, whose time has density $f_n(s) = \lambda^n s^{n-1} e^{-\lambda s} / (n-1)!$ for $s \geq 0$ by that section’s lemma “Arrival-time density”. Since $s f_n(s) = (n/\lambda) f_{n+1}(s)$ and f_{n+1} integrates to 1,

$$\mathbb{E}[S_n] = \int_0^\infty s f_n(s) ds = \frac{n}{\lambda} \int_0^\infty f_{n+1}(s) ds = \frac{nT_0}{p}. \quad \square$$

Table 3 evaluates Proposition 6.4 for every count of Table 2, against the attacker share for which that count was computed, taking $T_0 = 25$ seconds, the spacing produced when the full hashrate publishes. The nominal wait nT_0 understates each entry by the factor p : the 539 confirmations required against a 45% attacker take a nominal 3.74 hours, but 6.81 hours in expectation while that attacker withholds. A wait of hours spans many difficulty-averaging windows (Section 8.2), over which retargeting would lower the public chain’s difficulty; the proposition rests on Assumption 4.2 and omits that effect. Three confirmations take a nominal 75 seconds and ten take 250 seconds; a 10% withholding attacker divides these waits by 0.9.

Table 3. The confirmation counts n of Table 2 and their expected waits in minutes, nT_0/p with $T_0 = 25$ seconds by Proposition 6.4, while an attacker of share q withholds his blocks; computed by script.

q	$r < 10\%$		$r < 1\%$		$r < 0.1\%$	
	n	min	n	min	n	min
2%	1	0.4	2	0.9	3	1.3
5%	2	0.9	3	1.3	4	1.8
8%	2	0.9	3	1.4	5	2.3
10%	2	0.9	4	1.9	6	2.8
15%	3	1.5	6	2.9	9	4.4
20%	4	2.1	8	4.2	13	6.8
25%	5	2.8	12	6.7	20	11.1
30%	9	5.4	20	11.9	32	19.0
35%	15	9.6	36	23.1	58	37.2
40%	34	23.6	82	56.9	133	92.4
45%	135	102.3	331	250.8	539	408.3

A nominal thirty-minute wait yields 72 confirmations at 25-second spacing and three at Bitcoin’s 600-second spacing. The corresponding strict-overtake probabilities against a 10% attacker are approximately 9.35×10^{-34} and 1.712%, respectively; either count takes $30/0.9 \approx 33.3$ minutes in expectation under withholding. These are model calculations, not guarantees about either network.

The comparison holds q fixed. The model contains no network, so it omits the honest work lost when honest blocks race one another during propagation; quantifying that loss requires a propagation distribution and a mining topology. A policy that waits for a fixed amount of accumulated

miner reward measures a different quantity: if the reward per block decreases in proportion to the target spacing, reaching that amount requires proportionally more blocks, and, ignoring transaction fees (value paid for transaction inclusion) and rounding, the spacing cancels out of the mean wait. The monetary and fixed-hashrate models are therefore not interchangeable.

Remark 6.5 (Limits of the model). Three assumptions of the model fail on the deployed chain and limit how far the calculations transfer to it. (i) *Difficulty is not constant*: Zcash retunes the target after every block (Section 8.2). The race is scored in work, not block counts; the share q approximates the attacker's share of work production while the branches' block weights remain close, and once their difficulty schedules diverge, the equal-step random-walk formula is no longer exact. (ii) *Hashrate is not constant*: the share q in the theorems is the share the attacker sustains for the duration of the attack; the paper's caveat is that T_0 re-enters only if the attacker cannot sustain his hashrate long enough, which it judges unlikely for a serious attacker. (iii) *Propagation is not instantaneous*: honest blocks found concurrently compete during propagation, and the work of the losing block is wasted, which raises the attacker's effective share; the size of this effect is not quantified here. A fourth limitation lies outside the model: the attacker of this volume obeys the protocol's validity rules and attacks only the *choice* between valid histories. Other volumes treat attacks on the rules themselves.

7 The economics of the attack

The probabilities of Section 5 assume that the customer is an attacker. Following the paper's §6, this section derives the payment value below which the attack has non-positive expected profit (Proposition 7.1) and tabulates it in units of block income. Throughout, $q < p$ (“[o]therwise, all bets are off”, §6); the model is deliberately stylised, and Remark 7.3 records its caveats.

The attacker targets k merchants simultaneously with payments of v each, all invalidated by the same private branch; the goods are worth αv to him, $0 < \alpha \leq 1$; he abandons the attack after mining o blocks without success; and each of his blocks would earn the block income B if it were included in the accepted chain. If the attack succeeds, his blocks are all accepted, and he keeps the goods, the coins and the rewards. If it fails, he has paid kv , keeps goods worth $k\alpha v$, and his o blocks, excluded from the accepted chain (*orphaned*), forfeit oB . An exact profit calculation for this stopping rule would take r to be the success probability of that rule; following the paper, the table below substitutes the unlimited-horizon $r(q, n)$ as a heuristic proxy, and so does not evaluate exactly a strategy that stops after $o = 20$ private blocks.

Proposition 7.1 (Profitability; Rosenfeld §6). *Given success probability r and the gains and losses stipulated above, the attacker's expected profit over not attacking is*

$$k\alpha v - (1 - r)(kv + oB) = kv(\alpha + r - 1) - (1 - r)oB,$$

positive if and only if

$$v > \frac{(1 - r)oB}{k(\alpha + r - 1)}$$

(when $\alpha + r > 1$; when $\alpha + r \leq 1$ the modelled attack is never strictly profitable). The modelled attack is not strictly profitable whenever

$$v \leq \frac{o(1-r)B}{k(\alpha+r-1)} \stackrel{\alpha=1}{=} \frac{o}{k} \cdot \frac{B(1-r)}{r}, \quad (3)$$

which with the paper's choices $o = 20$, $k = 5$, $\alpha = 1$, $B = 25$ BTC is its equation (2), $v \leq 100(1/r - 1)$ BTC.

Proof. The gain kav is certain (the goods are obtained either way); the loss $kv + oB$ is paid exactly when the attack fails, with probability $1 - r$. Rearranging the positivity condition gives the threshold; substituting the paper's constants gives $20 \cdot 25 \cdot (1 - r) / (5r) = 100(1 - r)/r$. $\square$

7.1 Reward-normalised exposure

The *block subsidy* is value issued by the protocol in a block, distinct from fees transferred by its transactions. Its *scheduled* component is base issuance determined by height, before any reserve payout. A *funding stream* is a required allocation of a share of that subsidy to a designated recipient. The *coinbase* is the block's first transaction, which distributes the available subsidy and fees according to the applicable rules. The *reserve* is the accounting balance of funds removed from circulation and awaiting reissuance.

The income B forfeited by an orphaned block is the amount its miner would have retained: miner subsidy plus retained transaction fees, not the subsidy allocated to other recipients. Under NU7, from activation until the scheduled subsidy next halves, the miner subsidy is 41,666,668 zatoshi per block (Section 8.7), and the miner keeps the fees not removed from circulation, 40% rounded in the miner's favour; once reissuance begins, the subsidy also depends on the prior reserve balance (Section 8.8). A universal ZEC figure would therefore hide assumptions about fee income and chain state.

Instead, Table 4 measures transaction value in units of the chosen block income B . With the paper's $o = 20$, $k = 5$, and $\alpha = 1$, equation (3) becomes

$$\frac{v}{B} \leq 4 \frac{1-r}{r}.$$

Once a particular threat model supplies B , multiplying by it gives the corresponding monetary threshold. Treating B as constant over the attack remains a modelling assumption.

Within the model, a few confirmations make the attack unprofitable for an attacker with a small share at any plausible payment size; against a 30% attacker, six confirmations make modelled expected profit non-positive only up to about 21 times the assumed block income B . By the geometric decay of Theorem 5.2, the required count grows only logarithmically in the value at stake. These figures are neither exact finite-stop profit calculations nor bounds on deployed rollback risk; changing difficulty and rollback rails require separate modelling.

Remark 7.2 (Majority is a different problem). At $q = p$, eventual strict overtake has probability 1, but the walk has zero drift; that fact alone does not give the attacker a lasting monopoly. When $q > p$, the profit analysis does not apply: the paper notes that a strict majority attacker

Table 4. The largest whole-number multiple of block income B with non-positive heuristic expected profit: $\lfloor 4(1-r)/r \rfloor$, computed by script. The table is dimensionless: it neither fixes a ZEC subsidy nor predicts future fee income. Values inherit every assumption of the model; the caveats of Remark 7.3 apply in full.

q	$n=1$	2	4	6	8	10
5%	36	271	10,327	344,743	10,931,506	336,737,801
10%	16	67	729	6,759	59,475	508,917
20%	6	15	55	167	467	1,262
30%	2	5	11	21	35	57
40%	1	1	2	4	5	6

can double-spend at no cost beyond normal mining, reject every other miner’s blocks to take the whole issuance, and exclude transactions at will, driving honest miners out and entrenching his majority. A majority attack therefore “is best seen as an attempt to destroy” the currency, motivated from outside it (a short position, a competing system), not an attempt to profit inside it. No confirmation policy defends against it; the defence is the cost of assembling the majority, which lies outside this model.

Remark 7.3 (The model’s own caveats). The paper states each limitation of its model. The give-up point $o = 20$ “is a significant simplification” (an optimal attacker balances completion against compounding losses); the safe values “should be taken with a grain of salt, because of the many modeling assumptions”; and a model resting on mining *costs* rather than forfeited *rewards* would make the required confirmations linear, not logarithmic, in the transaction value — “very poor security, hence in a situation where this is relevant, we have already lost anyway” (§6). The bound is significant for its form, a required confirmation count logarithmic in the value at stake, not for its numerical precision.

8 NU7: 25-second blocks and bounded shielded work

This section describes the ZIP-259 draft, a proposal rather than a deployed upgrade, which combines four changes: shorter blocks with limits on shielded work (ZIP-218), removal of 60% of transaction fees from circulation (ZIP-235), reissuance that preserves the halving schedule — periodic divisions of scheduled subsidy by two (ZIP-237) — and rejection of version 4 transactions (ZIP-2003). Three amendments adjust the existing funding streams: Revision 3 of ZIP-214 rescales their end heights to the new spacing (Section 8.7), Revision 2 of ZIP-207 rescales the periods for which each recipient address is used, and ZIP-2008 changes one stream’s recipient address. The activation height, denoted A , must be a multiple of three (Revision 3 of ZIP-214). At the source boundary of Section 1.1, ZIP-259 sets $A = 4,465,026$ on Testnet and leaves the Mainnet value unassigned.

NU7 introduces no new transaction format: versions 5 and 6 remain available. Their signatures nevertheless commit to a *consensus branch identifier*, a value identifying the network upgrade’s rules. An unchanged encoding therefore does not make a pre-NU7 signature valid under NU7. Disabling version 4 also disables spending funds in Sprout, an older shielded pool, because the newer formats cannot represent its transfers; it does not transfer their value to a reserve or remove

it from the supply.

8.1 Chain selection and the tie-break

The most-work rule of Definition 2.2, which compares the sum of the blocks' work rather than their count, is unchanged. Changing target spacing changes neither the work definition nor this ordering.

Remark 8.1 (The implementation tie-break). The specification and Rosenfeld's model prefer, at equal cumulative work, the tip received first. Zebra applies the same rule from version 6.4.0, released on 23 September 2026: of two chains with equal work it keeps the one whose tip it received first, and compares tip hashes only between tips whose receipt times it cannot order. A tying branch that reaches a node after the public tip therefore does not displace it there, as the strict-overtake boundary $z = -1$ assumes. Earlier versions broke every equal-work tie by the larger tip hash. Under that rule an attacker could release a tying private branch whose tip hash won; the boundary $z = -1$ does not count that success, so for those versions $r(q, n)$ is a lower bound when only the tie-break is replaced, the other assumptions held fixed. Under either rule the tie-break concerns local chain choice, not the validity of a block, and $r(q, n)$ remains exact only for the stated model: changing difficulty and rollback limits also change the deployed success event (Remarks 8.2 and 8.3).

8.2 Difficulty adjustment

Assumption 4.2 fixes difficulty; Zcash retargets after every block. In ZIP-218, the NU7 target spacing is 25 seconds and the target-averaging window is 102 blocks. The timestamps determining the adjustment still come from the branch being evaluated, including on an attacker's private branch.

Write $N = 102$ for the target-averaging window and $T = N \cdot 25 = 2,550$ seconds for its nominal timespan.

The specification's §7.7.3 defines the rest of the calculation. For a block height b , let $m(b)$ be the median timestamp of the eleven preceding blocks: their sixth-smallest timestamp. Let $\bar{t}(b)$ be the mean target of the N blocks preceding b : the sum of their targets divided by N . Define the observed and damped timespans by

$$\begin{aligned} a(b) &= m(b) - m(b - N), \\ d(b) &= T + \text{trunc}\left(\frac{a(b) - T}{4}\right), \end{aligned}$$

where trunc discards the fractional part towards zero. The bounded timespan $c(b)$ is $d(b)$ clamped between $[0.84T]$ and $[1.32T]$: values below the lower endpoint are replaced by that endpoint, and similarly above the upper endpoint. The maximum target PoWLimit is $2^{243} - 1$ on Mainnet and $2^{251} - 1$ on Testnet (§5.3). The target calculation is

$$t(b) = \min\left(\text{PoWLimit}, \left\lfloor \frac{\bar{t}(b)}{T} \right\rfloor c(b)\right),$$

followed by the compact encoding defined in the specification's §7.7.4. The order of the floor and multiplication matters. At NU7 the clamp endpoints are 2,142 and 3,366 seconds; the eleven-timestamp median, fourfold damping, and target cap are unchanged.

Activation does not reset the accumulated work or multiply the target by three in a single operation. The averaging window of the first NU7 blocks still covers pre-NU7 history and adjust through the same bounded rule. Likewise, the timestamp rules do not shrink with the target spacing: a timestamp must exceed the median of the preceding eleven and be at most 90 minutes above it (§7.6). A further check, not a consensus rule, rejects received blocks more than two hours ahead of the validator’s clock. Testnet’s exceptional minimum-difficulty rule sets a block’s target to PoWLimit when its timestamp exceeds its predecessor’s by strictly more than a threshold. At the Testnet NU7 activation height the threshold changes from six to eighteen target spacings, so it remains $18 \cdot 25 = 6 \cdot 75 = 450$ seconds: a gap of 450 seconds does not qualify, one of 451 seconds does (ZIP-218, §“Minimum difficulty blocks on Testnet”). This exception does not apply to Mainnet.

Remark 8.2 (What per-block retargeting changes). The averaging window, damping, and clamp constrain the adjustment but do not prove that two forked branches retain close targets. Work per block moves inversely with the target; a branch’s timestamps can change both its future block rate and the work carried by each block. Equal expected work production does not make the probability of an eventual overtake independent of these jump sizes. Consequently, the probability tables are an equal-work baseline, not an exact analysis of private branches whose difficulty schedules diverge.

8.3 Equihash and the memoryless model

Zcash’s proof of work is Equihash with parameters $n = 200$, $k = 9$ (specification §7.7, §7.7.1; the algorithm is Biryukov and Khovratovich’s, ePrint 2015/946), but the difficulty test a block must also pass is a separate later step: “[t]he difficulty filter is unchanged from Bitcoin, and is calculated using SHA-256d on the whole block header (including solutionSize and solution)”, required to be at most $\text{ToTarget}(\text{nBits})$ (§7.7.2). Each candidate Equihash solution therefore yields a new header whose SHA-256d hash is tested against the target. Treating distinct headers as independent trials is the usual random-function model for SHA-256d, not a property asserted by the specification; under that model these are the Bernoulli trials of Remark 3.7.

The specification nowhere asserts progress-freeness — the argument is this volume’s, not a citation. What Equihash changes is the granularity: candidates arrive in batches, one batch per memory-hard solver run. Progress within a run is discarded when a competing block arrives. Under independent identically distributed runs, the number of runs until an accepted header is geometric; replacing those discrete completion times by an exponential clock is a further approximation, coarse within one run and good only when a run is short relative to the target spacing: 25 seconds in the NU7 draft. Run duration is implementation- and hardware-dependent and is not quantified here. A proof of work whose solve time is comparable to the spacing would make block discovery visibly non-exponential and move the attribution sequence away from that of Proposition 3.4.

8.4 Rollback limits, coinbase maturity, and finality

Consensus does not impose a maximum reorganisation depth. ZIP-218 recommends a node-local 600-block rollback window: at the target spacing this is $600 \cdot 25 = 15,000$ seconds, or 250 minutes.

Such a limit bounds the history that a node automatically reverses; it is not a proof that older blocks cannot be replaced by a higher-work valid chain. Following a branch outside that retained history requires different local state or operator intervention. Actual node implementations may choose a different window.

Remark 8.3 (Effect of a rollback limit). A finite rollback policy truncates some paths to eventual overtake, so the unlimited-rollback probability $r(q, n)$ is not the exact failure probability of a node enforcing that policy. A limit is also not global agreement: different nodes may stop on different sides of a long split. The specification’s §3.3 adds a separate requirement for validators that risk Mainnet funds or display transaction information: their chain must include the activation block of the most recent settled network upgrade. That socially anchored checkpoint is distinct from probabilistic finality.

A *transparent output* records its value and destination in cleartext. The consensus maturity rule prevents spending such an output of a coinbase until the spending block is at least 100 heights later (§7.1.2). ZIP-218 keeps that count, a nominal $100 \cdot 25 = 2,500$ seconds. Coinbase outputs placed directly in a shielded pool do not have this transparent maturity restriction. A 600-block rollback window can therefore reverse an already mature coinbase: maturity permits spending, not finality.

In the model, maturity is a confirmation count fixed by consensus. A spend can enter a block only once the coinbase’s block has 100 confirmations on the chain that the spend extends. A private branch forked below the coinbase block, under the conventions of Theorem 5.2, therefore removes a newly mature output from the best chain with probability $r(q, 100)$. Computed by script, $r(0.3, 100) \approx 3.68 \times 10^{-9}$, $r(0.4, 100) \approx 0.432\%$, and $r(0.45, 100) \approx 15.7\%$. The count is ample against a 30% attacker and weak near one half, where, as for every fixed count, $r(q, 100)$ tends to 1 (proof of Proposition 6.2).

8.5 Wallet confirmation policy

A wallet can distinguish *trusted* outputs that its own transaction handling can recreate after a rollback from an *untrusted* external payment that may require the sender to pay again. The draft ZIP-315 recommends three confirmations for the former and ten for the latter, and exempts the shielding of transparent funds from its three-confirmation minimum. The examined wallet back-end follows both, allowing zero-confirmation shielding by default. These choices are not consensus rules.

Table 1 quantifies their modelled trade-off: three confirmations assign a 10% attacker risk 1.712%, while ten assign the same attacker about 0.0008% and a 20% attacker about 0.316%. The count remains a (q, ϵ) policy, not a guarantee.

The *anchor* is the note-commitment-tree root against which a spend proves membership (*Crypto Guide*, §Privacy-preserving membership via zero-knowledge paths). ZIP-218 keeps at three blocks the anchor depth that ZIP-315 recommends. ZIP-218 also recommends a transaction-expiry delta of 120 blocks, a nominal $120 \cdot 25 = 3,000$ seconds, without overriding an explicitly configured expiry delta. ZIP-218 does not prescribe dividing every confirmation count by three.

Table 5 collects the safeguards of Sections 8.2 to 8.5. Four are consensus rules, which decide block validity; the rollback window is node policy, and the confirmation counts and anchor depth

are wallet policy, which govern only a node’s or a wallet’s own conduct. Every value is specified, in the protocol specification or in a draft ZIP at the source boundary of Section 1.1; none is designed but unspecified.

Table 5. The NU7 safeguards. Durations are nominal, at 25 seconds per block (the draft ZIP-218 spacing), and computed by script; § numbers are the protocol specification’s; “draft” marks a ZIP of Draft status.

Safeguard	Value	At 25 s	Standing	Model assumption
Difficulty adjustment	every block; window $N = 102$ (17 before NU7); damping 4; clamps $[0.84T]$ and $[1.32T]$	$T = 2,550$ s; clamps 2,142 and 3,366 s	consensus, §7.7.3; $N = 102$ draft ZIP-218	constant difficulty (Assumption 4.2), replaced (Remark 8.2)
Timestamp bounds	above the median of the 11 preceding timestamps; at most 90 min above it	216 spacings	consensus, §7.6	constant difficulty: bounds the timestamps that set a branch’s targets
Proof of work	Equihash (200, 9), then SHA-256d header hash at most the target	—	consensus, §7.7.1, §7.7.2	exponential clocks (Remark 3.7), coarsened by solver runs
Rollback window	600 blocks	250 min	node policy: §3.3 permits one; draft ZIP-218	unlimited rollback (Remark 8.3)
Coinbase maturity	100 blocks, transparent outputs	41.7 min	consensus, §7.1.2; kept by draft ZIP-218	none; fixes $n = 100$
Wallet confirmations	3 trusted, 10 untrusted	75, 250 s	wallet policy, draft ZIP-315	none; a choice of (q, ε)
Anchor depth	3 blocks	75 s	wallet policy, draft ZIP-315 and ZIP-218	none; outside the race

8.6 A block budget for shielded work

A byte limit does not directly bound the computational work inside a block. The 2,000,000-byte limit must therefore be considered alongside the 25-second target spacing and the costs of shielded validation and wallet scanning. ZIP-218 adds limits on the shielded components that drive this work.

An Orchard *Action* pairs one spend slot with one output slot; dummy slots allow either side to carry no real transfer. Ironwood is the newer shielded pool using the same Action structure (ZIP-229). The older Sapling shielded pool counts spends and outputs separately; its cryptographic construction is not needed for this accounting.

Let O and I count Orchard and Ironwood Actions in a block, let S count Sapling spends plus outputs, and let J count Sprout transfers (JoinSplits), each of which creates two notes. ZIP-218 bounds each pool and charges all of them to one shared budget, GlobalShieldedBudget = 330:

$$O \leq 330, \quad I \leq 330, \quad S \leq 300, \quad J \leq 0, \quad O + I + S + 2J \leq 330.$$

The final inequality is shared across pools: their separate maxima cannot all be filled in one block. For example, $O = 100$, $I = 200$, $S = 30$ exhausts the shared budget; increasing I to 201 violates it even though each per-pool limit still holds. The Sprout bound of zero restates what the rejection of version 4 already enforces, since only that format represents Sprout transfers.

At two Actions per transaction, Orchard and Ironwood transactions together fit at most 165 per block under this budget, or a nominal $165/25 = 6.6$ transactions per second. This is a capacity calculation under a specified transaction shape, not a measured throughput guarantee. Transparent components remain subject to the byte limit rather than this shielded-action budget.

8.7 Scheduled issuance with shorter blocks

A *zatoshi* is the atomic currency unit: 1 ZEC equals 10^8 zatoshi. A *halving* divides the scheduled subsidy by two. Retaining its nominal wall-clock interval while tripling block frequency requires both three times as many blocks per interval and one third of the subsidy per block. ZIP-218 therefore changes the full interval from 1,680,000 to $L = 5,040,000$ blocks. These earlier intervals are needed to compute progress at activation, not to supply an alternative rule. The change preserves expected timing, not exact calendar dates, because actual block times remain random.

The halving count after activation retains the progress already made towards the next halving. Let H_B be the Blossom activation height: 653,600 on Mainnet and 584,000 on Testnet. Before Blossom, halvings were separated by 840,000 blocks; the initial slow-start schedule, which ramps up the subsidy during the first 20,000 blocks, contributes a shift of 10,000 blocks. These constants and the maximum subsidy $M = 1,250,000,000$ zatoshi are defined in the specification's §5.3 and §7.8. For a height $b \geq A$, the draft's halving count and scheduled subsidy s_b , measured in zatoshi, are

$$k(b) = \left\lfloor \frac{H_B - 10,000}{840,000} + \frac{A - H_B}{1,680,000} + \frac{b - A}{5,040,000} \right\rfloor, \quad (4)$$

$$s_b = \left\lfloor \frac{M}{2 \cdot 3 \cdot 2^{k(b)}} \right\rfloor. \quad (5)$$

The factors 2 and 3 account separately for Blossom's and NU7's spacing changes. If activation occurs during the second-halving era, $k(b) = 2$ gives $s_b = 52,083,333$ zatoshi, or 0.52083333 ZEC, until the next halving; $k(b) = 3$ gives 26,041,666 zatoshi.

The old Mainnet third-halving height $H_3 = 4,406,400$ is therefore not the NU7 third-halving height if $A < H_3$. Converting the remaining interval gives

$$H_3^{\text{NU7}} = A + 3(H_3 - A) = 13,219,200 - 2A.$$

On Testnet, where $H_3 = 4,476,000$ and $A = 4,465,026$, the same conversion gives $H_3^{\text{NU7}} = 4,497,948$; the Mainnet value awaits the Mainnet activation height.

Retiming the halving function does not by itself retime a funding stream whose end is a literal block height. Revision 3 of ZIP-214 supplies that change: the two streams that end at the old third halving, of 8% and 12% of the block subsidy, end at H_3^{NU7} instead. Each stream receives the floor of its share, so at $s_b = 52,083,333$ they receive 4,166,666 and 6,249,999 zatoshi, and the miner subsidy, the remainder (specification §7.8), is 41,666,668 zatoshi per block.

8.8 The NSM reserve and delayed reissuance

The *Network Sustainability Mechanism* (NSM) separates temporarily removed funds from the still-unissued scheduled subsidy. In ZIP-237, the *issued supply* V_b is the total value recorded in the chain's value pools after block b . Each pool is a separately tracked balance for one part of the protocol; tracking a shielded pool's total does not reveal the values of its individual notes. The *NSM reserve* R_b , which ZIP-237 calls the NSM Value Balance, records value removed from circulation and not yet reissued. The reserve is public consensus state, not a spendable pool or an account with a spending key. In particular, it is not 21 million ZEC minus the issued supply: that larger difference also includes future scheduled issuance.

The initial reserve recovers the historical subsidy and fees that miners left unclaimed before NU6. Since ZIP-236, a coinbase must account for its full available value, so that historical shortfall is fixed. The draft defines the seed R_{A-1} as total scheduled issuance through the last pre-NU6 block minus issued supply there, and states the measured values: 36,858,445,520 satoshi (about 368.58 ZEC) on Mainnet, through height 2,726,399, and 55,768,414,957 satoshi on Testnet, through height 2,975,999.

Let d_b be the number of satoshi removed from circulation in block b . NU7 deploys one removal mechanism, ZIP-235; ZIP-259 excludes the voluntary removal of ZIP-233. With F_b the total fee paid by the non-coinbase transactions of block b ,

$$d_b = \left\lfloor \frac{6F_b}{10} \right\rfloor,$$

computed once from the block's total fee. The coinbase's total input value (specification §7.1.2) counts $F_b - d_b$ in place of F_b , and since ZIP-236 requires the coinbase to claim exactly its total input value, the miner receives exactly $F_b - d_b$; no transaction field states d_b . The rounding favours the miner and is applied once: two fees of 1 satoshi remove $\lfloor 12/10 \rfloor = 1$ satoshi, where rounding each transaction's share would remove none.

With $L = 5,040,000$ blocks per halving interval, the draft selects the reissuance fraction

$$f = \frac{\lfloor 6,931,680,000/L \rfloor}{10^{10}} = \frac{1375}{10^{10}}.$$

Reissuance starts at a height D that ZIP-237 derives from the scheduled subsidy alone, independently of actual removals. Let C_b be cumulative scheduled issuance through height b , with the pre-NU7 subsidies below A and s_b from A , and let H_3^{NU7} and H_4^{NU7} be the heights at which $k(b)$ of equation (4) first reaches 3 and 4. Then D is the least b with $\max(A, H_3^{\text{NU7}} + 1) \leq b < H_4^{\text{NU7}}$ and

$$\lfloor f(2.1 \cdot 10^{15} - C_{b-1}) \rfloor < s_b,$$

the first block at which reissuing the fraction f of the cap's unissued remainder, rounded up, would pay less than the scheduled subsidy; the cap $2.1 \cdot 10^{15}$ satoshi is 21 million ZEC. In that range $s_b = 26,041,666$ is constant, so C grows by that amount per block and the condition reads

$$2.1 \cdot 10^{15} - C_{b-1} \leq \left\lfloor \frac{26,041,665 \cdot 10^{10}}{1375} \right\rfloor = 189,393,927,272,727.$$

Let H_2 be the pre-NU7 second-halving height: 2,726,400 on Mainnet and 2,796,000 on Testnet. If A lies between H_2 and the pre-NU7 third halving H_3 , the remainder after block H_3^{NU7} is $262,499,975,638,334 - (A - H_2)$ zatoshi, and for every multiple of three in that range the condition first holds 2,807,274 blocks after H_3^{NU7} , both figures computed by script. Hence

$$D = H_3^{\text{NU7}} + 2,807,274,$$

about 2.2 years of 25-second blocks after the third halving: 7,305,222 on Testnet, as ZIP-259 records, and $16,026,474 - 2A$ on Mainnet, unassigned with A .

The additional subsidy u_b , total block subsidy b_b , and reserve update are

$$u_b = \begin{cases} 0, & b < D, \\ \lfloor fR_{b-1} \rfloor, & b \geq D, \end{cases} \quad (6)$$

$$b_b = s_b + u_b, \quad R_b = R_{b-1} - u_b + d_b. \quad (7)$$

All amounts are integers in zatoshi. Reissuance uses the previous block's reserve, so a removal in the current block cannot finance its own additional subsidy. Before D , removals accumulate without payout. After D , a positive reserve pays at least one zatoshi: because R is an integer and $0 < f < 1$, one has $1 \leq \lfloor fR \rfloor \leq R$ for $R > 0$. The draft rejects a negative reserve; it does not silently clamp it to zero.

For a small exact example, take $R_{b-1} = 100,000,000$ zatoshi (1 ZEC) and $b \geq D$. The additional subsidy is $\lfloor 13.75 \rfloor = 14$ zatoshi. Without a new removal the reserve becomes 99,999,986; with a same-block removal of another 1 ZEC it becomes 199,999,986, but the current payout remains 14. Because no payout precedes D and $d_b \geq 0$, the seed bounds the first payout from below: $\lfloor 1375 \cdot 36,858,445,520 / 10^{10} \rfloor = 5,069$ zatoshi on Mainnet and 7,669 on Testnet. Fees removed between A and D raise it.

Why this preserves supply and halvings. Transfers between ordinary value pools cancel in their total. A block increases that total by its subsidy and reduces it by its removals:

$$V_b - V_{b-1} = s_b + u_b - d_b, \quad (V_b + R_b) - (V_{b-1} + R_{b-1}) = s_b.$$

The second identity follows by adding the reserve update; reissuance moves value between the two ledgers without creating a second copy. Scheduled issuance retains its halvings, with the reserve paying an additional amount on top. Funding streams, wherever active, take their prescribed fractions of the *total* subsidy b_b , not only s_b . The resulting subsidy depends on prior chain state as well as height.

Without integer rounding and new removals, the recurrence becomes $R_{b+j} = (1 - f)^j R_b$, so keeping about half the reserve after L blocks requires $f = 1 - 2^{-1/L} \approx (\ln 2)/L$. The draft implements an integer-scaled approximation with numerator 1375: $(1 - 1375/10^{10})^{5,040,000} \approx 0.50007$. This is a half-life of the reserve without new contributions, not a new halving rule for the scheduled subsidy; small reserves depart from the smooth curve because every positive payout rounds up.

9 Layer boundary

The theorems of this volume use, from the *Math Guide*'s probability section, the definition of probability, its extension beyond finite spaces, and the continuity theorem used in the catch-up proof. This volume supplies the best-chain model and the quantitative bounds in Tables 1, 2, and 4. Policies that decide which view of the chain to trust, or when to treat a transaction as final, belong to the layers above this volume.